Contents

1 Introduction **2**

2 The structure of the data **3**

3 Data preparation **3**

3.1 Calculating the probability of landing a trick for each contender. 5

3.2 A scatter plot of the scores on run 1 and run 2 6

4 A frequentest model **6**

4.1 Finding a point estimate for θ for each contender 7

4.2 Finding a point estimate for $[\alpha_i, \beta_i]^{\mathrm{T}}$ for each contender. 8

5 Finding a model for Y_i **11**

6 Simulating LCQ's **12**

7 A bayesian model **13**

7.1 "Generating 5,000 random outcomes from the posterior distribution"

$$f_{\theta_i,\alpha_i,\beta_i|\boldsymbol{X}_i}\left(\theta_i, \alpha_i, \beta_i \mid \boldsymbol{x}_i\right)$$

. 15

8 Sources **15**

A statistical and probabilistic analysis of 2022 Street League Skateboarding (SLS) Super Crown Championship

Jonathan Tadesse

1 Introduction

This study aims to construct a predictive model for determining finalists in the 2022 Street League Skateboarding (SLS) Super Crown Championship, a notable event in the realm of professional skateboarding.

The championship structure includes a Last Chance Qualifier (LCQ) followed by the primary final round. Within the final, eight positions are available, with four already occupied by skateboarders Horigome, Joslin, Milou, and Ribeiro G. based on their cumulative seasonal performance.

The LCQ encompasses sixteen participants, competing for the remaining four slots in the final round. The LCQ participants are:

Majerus	Oliveira	Decenzo	Santiago Papa
Eaton	Mota	Shirai Jordan	Hoefler
Hoban	Gustavo Ribeiro C	O'neill	Foy
Midler.			

In the LCQ, each skateboarder undertakes two 45-second runs and is allocated four single trick attempts. Runs and tricks are scored on a scale of 0 to 10, with a trick not successfully landed receiving a score of zero. The aggregate score for a skateboarder is the summation of their two highest trick scores and their peak run score. The top four from the LCQ progress to the championship final.

For this investigation, data from three additional events of the 2022 season have been analyzed. Each event adheres to a specific format, wherein skateboarders are assigned a sequential order for activities, encompassing: run 1, run 2, trick 1, trick 2, trick 3, and trick 4. This order is uniformly observed for all participants.

The qualifiers are formatted similarly to the LCQ, but participants are grouped into distinct heats. In the final, the leading eight scorers from the qualifiers proceed. During the final, after an initial round of two runs and four tricks and subsequent scoring, the bottom four are eliminated. The remaining participants are given two additional trick attempts, post which scores are reassessed, leading to the declaration of the overall winner.

This study provides an in-depth analysis of the competition structure, participant performance, and data-derived predictions.

2 The structure of the data

The data is presented in a csv file and contains 14 columns namely ,

Name	Data Type	Description
id	string	skateboarder's last name
location	string	location of the competition
month	integer	competition month
year	integer	competition year
comp	string	type of competition ('prelim men' or 'final men')
heat	integer	Heat in which the skateboarder participated (heat = 1 if comp = 'final men')
run 1	float	score for run 1
run 2	float	score for run 2
trick 1	float	score for trick 1
trick 2	float	score for trick 2
trick 3	float	score for trick 3
trick 4	float	score for trick 4
trick 5	float	score for trick 5 ('NaN' if eliminated after trick 4)
trick 6	float	score for trick 6 ('NaN' if eliminated after trick 4)

Table 1: Description of Skateboarding Data Fields

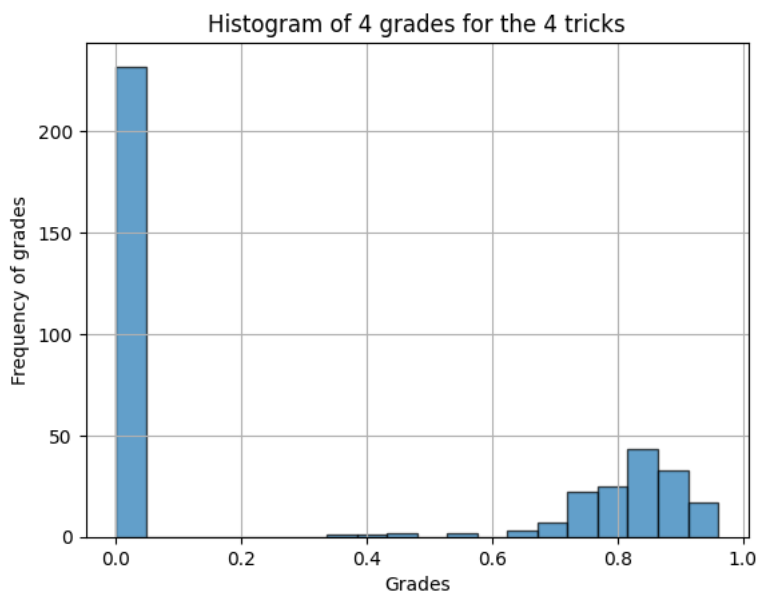
3 Data preparation

We begin by creating a dataframe for the content in the csv file using the module pandas in python. For practical purposes we normalize each entry point in the columns run 1-run 2 and trick 1-trick 6 so that they are within the bound [0.1],

```
"Read the file"
df = pd.read_csv("SLS22.csv")
print(df.to_string())

competition_columns=["run 1", "run 2", "trick 1", "trick 2", "trick 3",
    "trick 4", "trick 5", "trick 6"]

for column in competition_columns:
  df[column]=df[column].multiply(0.1) "multiply with 0.1 since each
      score is within 0-10"
```

Creating a histogram of all entries for the tricks 1-4 can help us see the frequency and distribution of different scores which might be useful in future tasks.

```
"Convert the 4D dimensional array into a 1D array and use matplotlib to
    plot the hisotogram."
grades = df[['trick 1', 'trick 2', 'trick 3', 'trick
    4']].values.flatten()

# Plot histogram
plt.hist(grades, bins=20, edgecolor='k', alpha=0.7)
plt.xlabel('Grades')
plt.ylabel('Frequency of grades')
plt.title('Histogram of 4 grades for the 4 tricks')
plt.grid(True)
plt.show()
```



The graph above seem to indicate that most contenders failed to hit their tricks, which explains the bar centered at grade $= 0$. For the other grades within the boundary $[0, 1]$, they don't seem to be uniformly distributed, but rather cluster around $[0.6, 0.9]$, the grades in this interval seem to follow a normal distribution.

3.1 Calculating the probability of landing a trick for each contender.

In the subsequent step, we compute the probability of each contender receiving a score $Z > 0.6$ given that they successfully executed the trick.

To calculate the desired probability, let's define two events:

S : the event that a contender scored higher than 0.6,

L : the event that the contender successfully landed the trick.

Using Bayes' theorem, the conditional probability $P(S \mid L)$ is given by:

$$P(S \mid L) = \frac{P(L \mid S) \cdot P(S)}{P(L)}$$

Now, given that if a contender scored higher than 0.6 (event S), they definitely landed the trick (event L), we have:

$$P(L \mid S) = 1$$

Substituting this into our equation, we get:

$$P(S \mid L) = \frac{P(S)}{P(L)}$$

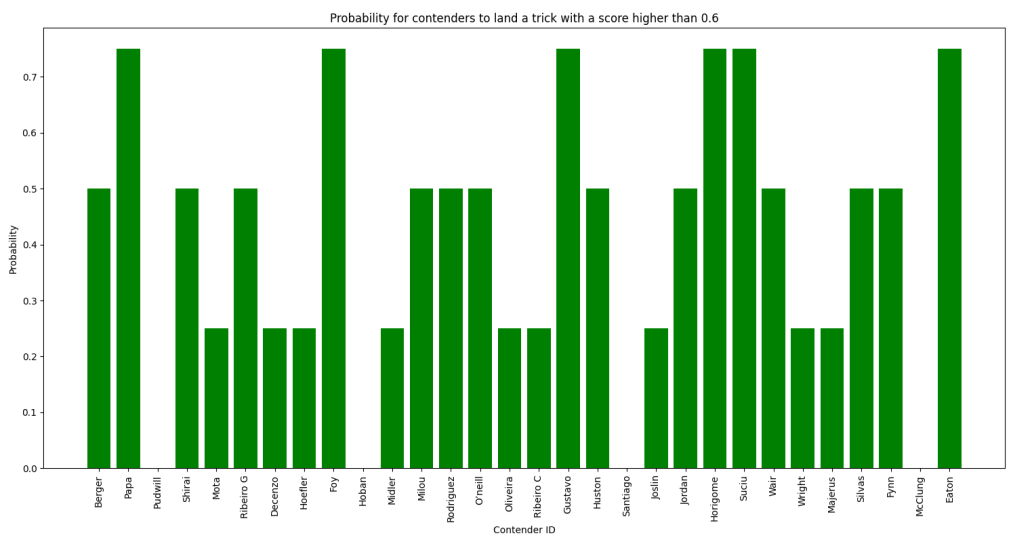
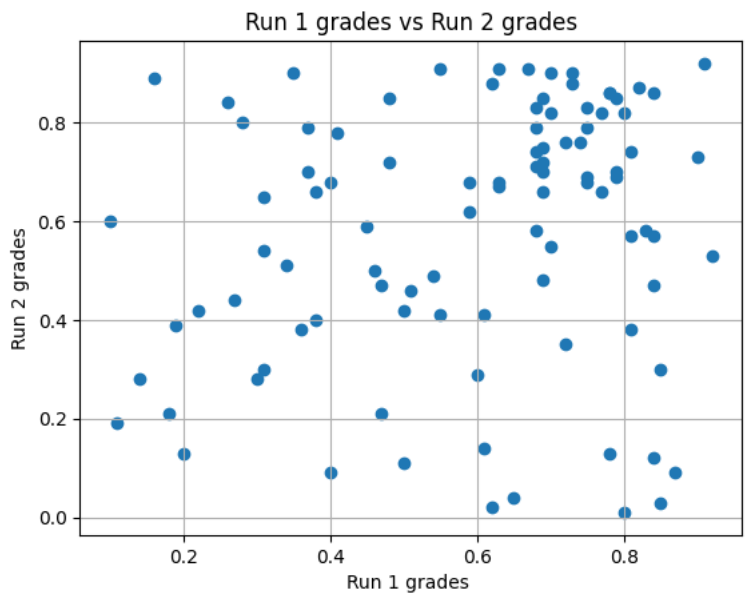


Figure 1: Probability for contenders to land a trick with a score higher than 0.6

The histogram seem to indicate an even spread of probabilities where the average probability θ is around 0.4.

3.2 A scatter plot of the scores on run 1 and run 2

In the next step a scatter plot between the columns run 1 and run 2 was made to visualize any correlations. The plot doesn't seem to indicate a correlation between the two columns run 1 and run 2 up until the point $[0.6, 0.6]$, but after this point the grades seem to cluster, perhaps indicating that contenders who did well on one run also did well on the other run.



4 A frequentest model

A frequentist model. We would like to build a model that can predict which of the 16 skateboarders in the LCQ will secure a place in the finals. One way to do this is to build a model for each skateboarder, use the models to simulate run scores and trick scores for each skateboarder, and combine the simulations to simulate the LCQ. We can simulate multiple LCQs and extract the four skateboarders with the highest total scores from each one. Our prediction becomes the mode of these results. Note that this model assumes that the performances of the skateboarders are independent. For simplicity, we assume that the score of a particular run Y_i and the score of a particular trick X_i are independent for each skateboarder i. We also assume that all trick scores and run scores are independent and identically distributed outcomes from X_i and Y_i respectively. We can then start by specifying a model for X_i and Y_i.

Based on the observations in Task 1, a reasonable model for X_i is as follows:

$$X_i = \begin{cases} 0 & \text{if } V_i = 0 \\ Z_i & \text{if } V_i = 1 \end{cases}$$

where $V_i \sim \text{Ber}(\theta_i)$, $Z_i \sim \text{Beta}(\alpha_i, \beta_i)$ and $V_i \perp Z_i$. It can be shown that

$$f_{X_i}(x_i|\theta_i, \alpha_i, \beta_i) = (1-\theta_i)\mathbf{1}_{x_i=0} + \theta_i f_{Z_i}(z_i).$$

The choice $V_i \sim \text{Ber}(\theta_i)$ models that a skateboarder receives a score of 0 if and only if they do not successfully land the trick, while the choice $Z_i \sim \text{Beta}(\alpha_i, \beta_i)$ models that the score for a particular trick is the portion of the trick that was "perfect."

4.1 Finding a point estimate for θ for each contender

Consider a contender i where $V_i \sim \text{Ber}(\theta_i)$. Given the data set

$$[V_1, V_2, V_3, \ldots, V_n],$$

the likelihood function is represented as:

$$L(\theta) = \theta^{\sum_{j=1}^{n} v_i}(1-\theta)^{n-\sum_{j=1}^{n} v_i},$$

which simplifies to:

$$L(\theta) = \theta^{n\bar{v}}(1-\theta)^{n-n\bar{v}}.$$

Taking the natural logarithm of the likelihood function, we obtain:

$$\log(L(\theta)) = n\bar{v}\log(\theta) + (n - n\bar{v})\log(1-\theta).$$

Differentiating with respect to θ, we get:

$$\frac{d\log(L)}{d\theta} = \frac{n\bar{v}}{\theta} - \frac{n - n\bar{v}}{1-\theta}.$$

To find the maximum likelihood estimate, set the above derivative to zero, yielding:

$$\hat{\theta} = \bar{v}.$$

The second derivative of $\log(L)$ is:

$$\frac{d^2\log(L)}{d\theta^2} = -\frac{n\bar{v}}{\theta^2} - \frac{n - n\bar{v}}{(1-\theta)^2}.$$

Given that $0 < \theta, \bar{v} \leq 1$, both terms in the second derivative are negative, implying that $\frac{d^2\log(L)}{d\theta^2}$ is negative. Therefore, the maximum likelihood estimate for θ for each contender i is $\hat{\theta} = \bar{v}$. Plotting a histogram for each contender and their $\hat{\theta}$ we get.

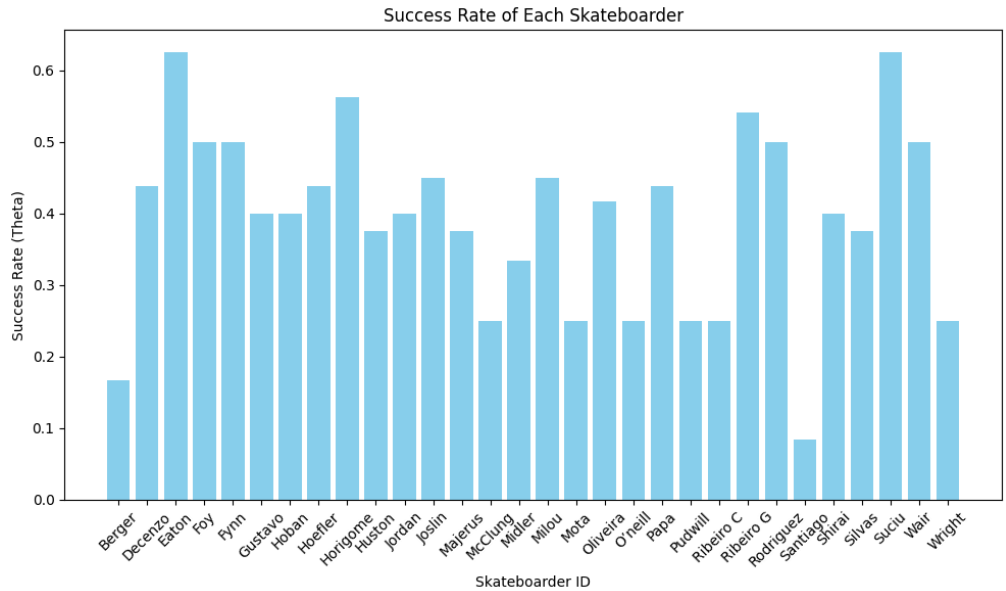


Figure 2: Succes rate for each contender

4.2 Finding a point estimate for $[\alpha_i, \beta_i]^{\mathrm{T}}$ for each contender.

To estimate the values for α and β, we can explore several options. One approach is to use the method of moments, which relies solely on sample data. Another strategy is the Maximum Likelihood Estimation (MLE) method, which incorporates both the data and the density function of Z_i. MLEs are typically preferred because they are consistent and asymptotically efficient, achieving the Cramér-Rao lower bound for variance as the sample size n approaches infinity, Whereas the method of moments may not always be asymptotically efficient or achieve the Cramér-Rao lower bound, we choose to use MLE due to this reason.

Because $Z_i \sim \text{Beta}\,(\alpha_i, \beta_i)$ the likelihood function becomes,

$$L(\alpha, \beta \mid \mathbf{Z}) = \prod_{i=1}^{n} \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} z_i^{\alpha-1} (1-z_i)^{\beta-1} = \left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)}\right)^n \prod_{i=1}^{n} (z_i)^{\alpha-1} \prod_{i=1}^{n} (1-z_i)^{\beta-1}$$

Taking the log of the likelihood function we get,

$$\begin{aligned}\log L(\alpha, \beta \mid \mathbf{Z}) &= n\log(\Gamma(\alpha+\beta)) - n\log(\Gamma(\alpha)) - n\log(\Gamma(\beta)) \\ &\quad + (\alpha-1)\sum_{i=1}^{n} \log(z_i) + (\beta-1)\sum_{i=1}^{n} \log(1-z_i)\end{aligned}$$

To find the point estimates for $[\alpha_i, \beta_i]^{\mathrm{T}}$ we need to take the partial derivatives

with respect to α and β and look at the stationary points for each derivative.

$$\frac{\partial}{\partial\alpha}\log L(\alpha,\beta \mid \mathbf{Z}) = \frac{n\Gamma'(\alpha+\beta)}{\Gamma(\alpha+\beta)} - \frac{n\Gamma'(\alpha)}{\Gamma(\alpha)} + \sum_{i=1}^{n}\log(z_i) = 0$$
$$\frac{\partial}{\partial\beta}\log L(\alpha,\beta \mid \mathbf{Z}) = \frac{n\Gamma'(\alpha+\beta)}{\Gamma(\alpha+\beta)} - \frac{n\Gamma'(\beta)}{\Gamma(\beta)} + \sum_{i=1}^{n}\log(1-z_i) = 0$$
$$z_i : z \neq 0, 1.$$

This system of equations is non-linear and does not have an analytical solution. As a result, we must use numerical methods to find stationary points. The Newton-Raphson method is suitable for this system because of it's fast convergence compared to other methods like Gradient Descent. The Newton-Raphson method however is sensitive to initial guesses and may not converge if the starting guess is far away from the stationary point. To mitigate this issue, we will use moment estimators to derive initial approximations for the parameters α and β. Since moments of method gives us a good initial approximation to the maximum we are certain that the potential maximum obtained from Newton-Raphsons will be the optimal estimate. First let us write out the Newton-Raphsons method.

To estimate the parameters $\hat{\theta} = (\hat{\alpha}, \hat{\beta})$ using the Newton-Raphson method, we utilize the following iterative equation:

$$\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n - J(\boldsymbol{\theta}_n)^{-1}\boldsymbol{F}(\boldsymbol{\theta}_n). \tag{1}$$

Upon normalization, the equations become:

$$f_1 = \psi(\alpha) - \psi(\alpha+\beta) - \frac{1}{n}\sum_{i=1}^{n}\log(z_i), \tag{2}$$

$$f_2 = \psi(\beta) - \psi(\alpha+\beta) - \frac{1}{n}\sum_{i=1}^{n}\log(1-z_i). \tag{3}$$

Let $\boldsymbol{F}(\boldsymbol{\theta}) = [\boldsymbol{f_1}, \boldsymbol{f_2}]$ and the Jacobian $J(\boldsymbol{\theta}_n)$ be defined as:

$$J(\boldsymbol{\theta}_n) = \begin{bmatrix} \frac{df_1}{d\alpha} & \frac{df_1}{d\beta} \\ \frac{df_2}{d\alpha} & \frac{df_2}{d\beta} \end{bmatrix},$$

where the entries are:

$$\frac{df_1}{d\alpha} = \psi'(\alpha) - \psi'(\alpha+\beta),$$
$$\frac{df_1}{d\beta} = -\psi'(\alpha+\beta),$$
$$\frac{df_2}{d\alpha} = -\psi'(\alpha+\beta),$$
$$\frac{df_2}{d\beta} = \psi'(\beta) - \psi'(\alpha+\beta).$$

The di-gamma and tri-gamma functions are introduced as:

$$\psi(\alpha) = \frac{\Gamma'(\alpha)}{\Gamma(\alpha)},$$
$$\psi'(\alpha) = \frac{\Gamma''(\alpha)}{\Gamma(\alpha)} - \frac{\Gamma'(\alpha)^2}{\Gamma(\alpha)^2}.$$

For an initial guess $\boldsymbol{\theta_0} = (\alpha, \beta)$, we derive the moments of estimators for α and β. Given the beta distribution, the moments are:

$$E(Z) = \frac{\alpha}{\alpha+\beta},$$
$$E(Z^2) = \frac{\alpha(\alpha+1)}{(\alpha+\beta)(\alpha+\beta+1)},$$
$$\mathrm{Var}(Z) = E(Z^2) - (E(Z))^2 = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}.$$

Using the method of moments, equating the sample mean $\bar{Z}$ and variance S^2, we obtain:

$$\alpha = \bar{Z}\left(\frac{\bar{Z}(1-\bar{Z})}{S^2} - 1\right),$$
$$\beta = (1-\bar{Z})\left(\frac{\bar{Z}(1-\bar{Z})}{S^2} - 1\right).$$

Thus, the method of moments estimators are:

$$\hat{\alpha}_{MOM} = \bar{Z}\left(\frac{\bar{Z}(1-\bar{Z})}{S^2} - 1\right) \quad \text{and} \quad \hat{\beta}_{MOM} = (1-\bar{Z})\left(\frac{\bar{Z}(1-\bar{Z})}{S^2} - 1\right).$$

To implement this in Python, we need to iterate through each contender listed in the "id" column and examine their scores in the columns for tricks 1-4, excluding scores that are 0. The aggregated scores for each contender are then used to obtain an initial guess, $[\alpha_0, \beta_0]$, for the Newton-Raphson method by leveraging the method of moments. However, some contenders have only one non-zero entry across the columns for tricks 1-4. This leads to a division by zero error in the calculation for $\hat{\beta}$ since we're dividing by the sample variance, which becomes 0. To address this, we identify contenders with fewer than 2 non-zero entries in the tricks columns and rely on the method of moments to estimate both $\hat{\alpha}$ and $\hat{\beta}$ using all their scores including 0.

An error tolerance of 10^{-6} and an iteration limit of 1000 is provided in the Newton-Raphson method to ensure convergence. The following table displays approximations of $[\alpha_0, \beta_0]$ and θ for each contender. The table seem to indicate that for most contenders $\hat{\alpha} > \hat{\beta}$ this means that the graph of the pdf for the contenders was shifted towards the positive z-axis. Which means that most contenders had high scores on average.

Theta, Alpha_Trick, and Beta_Trick values for each Skateboarder

Skateboarder	Theta	Alpha_Trick	Beta_Trick
Berger	0.166667	159.605769	59.031788
Papa	0.437500	22.444811	6.381090
Pudwill	0.250000	29.749604	19.531387
Shirai	0.400000	27.462426	3.157239
Mota	0.250000	23.685320	6.673465
Ribeiro G	0.541667	4.352759	1.091344
Decenzo	0.437500	23.828328	4.974541
Hoefler	0.437500	31.795658	9.186616
Foy	0.500000	51.899461	9.157862
Hoban	0.400000	109.713360	15.317418
Midler	0.333333	45.084903	10.750604
Milou	0.450000	36.877378	6.513038
Rodriguez	0.500000	54.426502	18.121561
O'neill	0.250000	1281.196150	238.011309
Oliveira	0.416667	77.462662	20.118766
Ribeiro C	0.250000	201.716043	58.010727
Gustavo	0.400000	66.358401	16.449841
Huston	0.375000	133.103941	17.008256
Santiago	0.083333	0.040903	1.003423
Joslin	0.450000	50.849552	8.606741
Jordan	0.400000	32.256032	5.126177
Horigome	0.562500	35.349729	3.782269
Suciu	0.625000	45.548883	13.737045
Wair	0.500000	12.664783	4.559670
Wright	0.250000	0.025000	0.113889
Majerus	0.375000	3.248495	2.947291
Silvas	0.375000	782.102890	179.496866
Fynn	0.500000	33.427072	7.010163
McClung	0.250000	0.109375	0.862847
Eaton	0.625000	73.099486	19.424381

Figure 3: $\hat{\alpha}$ and $\hat{\beta}$ for each contender

5 Finding a model for Y_i

Let Y_i denote the score for the ith run. Given that each run's score Y_i lies in the support interval $[0, 1]$, it is reasonable to model the variability in scores using a probability distribution that is defined on this interval. Considering the wide range of scores across contenders, a suitable choice is the Beta distribution.

The probability density function (pdf) of the Beta distribution is given by:

$$f(y; \alpha, \beta) = \frac{y^{\alpha-1}(1-y)^{\beta-1}}{B(\alpha, \beta)}$$

The reason Beta distribution is suitable is because of it's flexibility in representing a variety of shapes with just two parameters. For example, it can resemble:

- Uniform distributions when $\alpha = 1$ and $\beta = 1$.

- Bell-shaped (symmetric) distributions for $\alpha = \beta > 1$.
- Left-skewed distributions for $\alpha < \beta$.
- Right-skewed distributions for $\alpha > \beta$.
- U-shaped distributions for $0 < \alpha, \beta < 1$.

Like in the previous problem we will use Newton-Raphsons method to find the approximation to the MLE of α and β for each Y_i . Since no contender has received a 0 score on their runs we wont have to worry about having 0 variance when we find an initial guess for α and β using moment of methods. The following table displays $\hat{\alpha}$ and $\hat{\beta}$ for each contender.

Skateboarder	**Alpha**	**Beta**
Berger	2.675589	4.484919
Decenzo	4.297797	2.898543
Eaton	102.377807	36.435161
Foy	2.835828	3.512059
Fynn	5.604375	5.991282
Gustavo	1.565552	1.179158
Hoban	3.839819	2.192878
Hoefler	1.414922	1.021252
Horigome	15.069232	3.883174
Huston	7.611985	2.234350
Jordan	4.711139	1.715390
Joslin	11.930458	4.136985
Majerus	1.681258	2.480057
McClung	2585.982381	5892.648402
Midler	0.844620	0.801256
Milou	6.644679	2.876495
Mota	4.110897	4.464826
Oliveira	5.299018	3.887211
O'neill	0.909368	1.120608
Papa	2.359323	2.317255
Pudwill	4.905220	15.561172
Ribeiro C	2.067176	1.864905
Ribeiro G	6.252566	2.408003
Rodriguez	3.474851	3.788051
Santiago	2.763343	4.529392
Shirai	1.569434	1.090500
Silvas	0.821710	1.164738
Suciu	3.365321	1.873860
Wair	3.144349	2.266842
Wright	597.027030	1461.686984

Figure 4: $\hat{\alpha}$ and $\hat{\beta}$ for each $\boldsymbol{Y_i}$

We can see that the difference between $\hat{\alpha}$ and $\hat{\beta}$ for each contender isn't as big as it was for the trick scores. Indicating an even spread of the scores (bell shape of the pdf).

6 Simulating LCQ's

Utilizing the model for $[X_i, Y_i]^{\mathrm{T}}$, 5000LCQ simulations will be conducted. For each simulation, the top four skateboarders, represented as $\boldsymbol{W} = [W_1, W_2, W_3, W_4]^{\mathrm{T}}$,

will be extracted. The objective is to determine the most frequent winners across these simulations. For reference, the actual LCQ winners were Gustavo, Hoban, Eaton, and Decenzo. The analysis will also assess how accurately the simulations predicted these winners and calculate the estimated probability of these actual winners based on the simulations.
Each contender will complete two runs, but only the highest score from these runs will be considered. Additionally, each contender has four tricks to attempt. The cumulative score for each contender is the sum of their highest run score and their trick scores. The histogram below displays the most frequent winners based on 5,000 simulations. The top four most frequent winners were **Huston, Jordan, Shirai** and **Hoban**. The estimated probability of these being the actual winners, based on the simulations, was 0.06 percent.

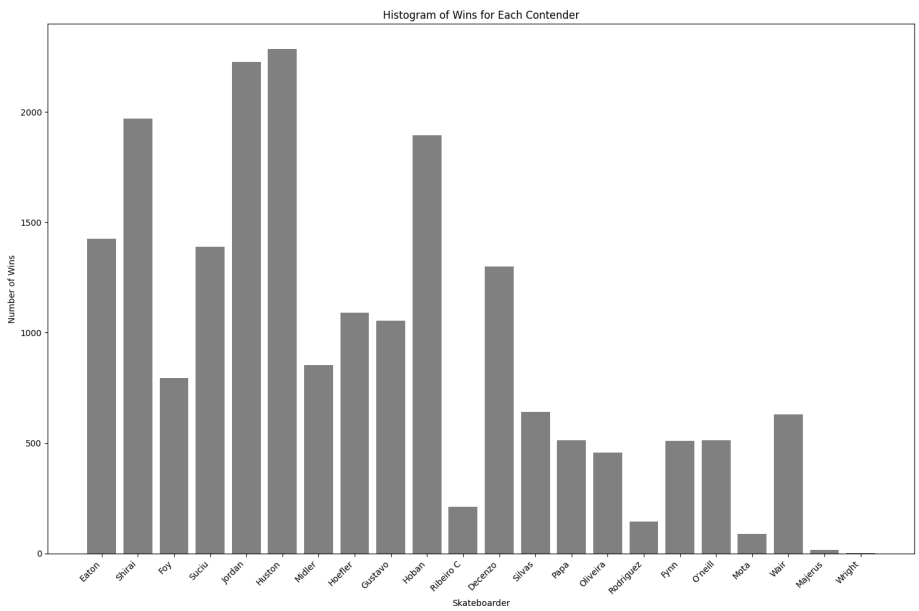


Figure 5: Most frequent winners of LCQ based on 5000 simulationsl

7 A bayesian model

As an alternative to the frequentist model, we could use a Bayesian model. Suppose we want to find the prior distribution for the parameters $[\Theta_i, A_i, B_i]^{\mathrm{T}}$ for X_i, assuming $\Theta_i \perp A_i, B_i$ for all i. Since θ is independent of A and B, the simultaneous prior can be expressed as $f_\theta(\theta) \times f_{\alpha,\beta}(\alpha, \beta)$. Starting with a prior for θ, we can assume a non-informative prior exist, so we get $f(\theta \propto 1$.

To find a conjugate prior for $[A_i, B_i]$, let's start by parameterizing α and β. The parameters in the Beta distribution are often viewed as shape parameters,

but their individual values may not carry a direct intuitive meaning. We aim to reparametrize the distribution to extract more meaningful information, such as the mean and the precision of the distribution which also will simplify the process of specifying the prior for the hyper-parameters. So let $\mu = \frac{\alpha}{\alpha+\beta}$ and $\kappa = \alpha+\beta+1$. We can now find a apriori distribution for $[\mu, \kappa]^{\mathrm{T}}$ and then apply a transformation to get the corresponding distribution for $[\alpha, \beta]^{\mathrm{T}}$. We choose the hierarchical model $\begin{aligned} \kappa \mid \mu &\sim \mathrm{Gamma}(\theta, \lambda), \\ \mu &\sim \mathrm{U}(0,1). \end{aligned}$

By applying the transformation we get

$$f_{A,B}(\alpha, \beta) = \frac{\lambda^{\theta}}{\Gamma(\theta)}(\alpha+\beta+1)^{\theta-1}e^{-\lambda(\alpha+\beta+1)}(\alpha+\beta)^{-1}$$

We need to make sure that the posterior distribution doesn't have it's maximum around $[0, 0]$, so we need to make appropiate choices for the parameters κ and λ. Let's first write the likelihood function and the joint posterior.

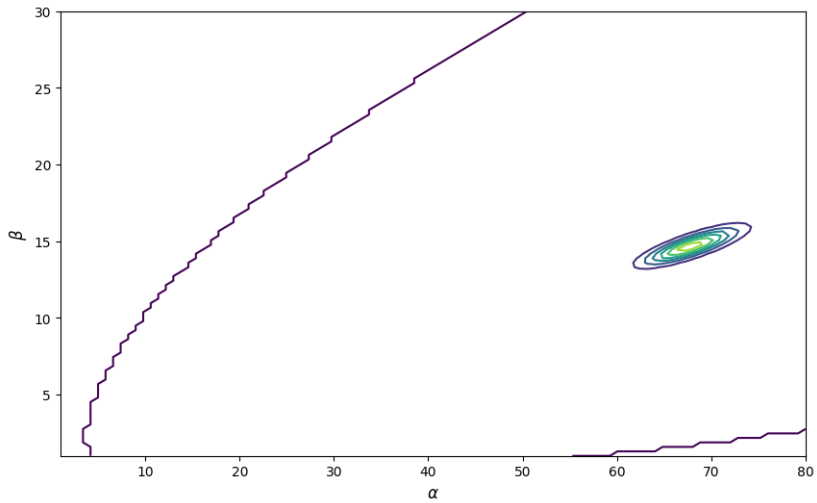
$$f_{X_i}\left(x_i \mid \theta_i, \alpha_i, \beta_i\right) = \prod_{i=1}^{n}((1-\theta_i)\,\mathbf{1}_{x_i=0} + \theta_i \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)+\Gamma(\beta)} z_i^{\alpha-1}(1-z)^{\beta-1}\mathbf{1}_{x_i\neq 0})$$

for $i = 1, 2, 3.....n$.

The joint posterior then becomes,

$$\begin{aligned} &f_{\Theta_i, A_i, B_i \mid \boldsymbol{X}_i}\left(\theta_i, \alpha_i, \beta_i \mid \boldsymbol{x}_i\right) \propto \\ &1 * \prod_{i=1}^{n}\left((1-\theta_i)\,\mathbf{1}_{x_i=0} + \theta_i \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)+\Gamma(\beta)} z_i^{\alpha-1}(1-z)^{\beta-1}\mathbf{1}_{x_i\neq 0}\right) \times \\ &\frac{\lambda^{\theta}}{\Gamma(\theta)}(\alpha+\beta+1)^{\theta-1}e^{-\lambda(\alpha+\beta+1)}(\alpha+\beta)^{-1} \end{aligned} \tag{4}$$

If we fixate $\lambda = n$ and let $\theta = \kappa\lambda = (j)(n) = t$ By suitable choices for the parameters we can ensure that the counterplots don't focus around $[0, 0]$. After experimenting with the values of λ and κ a suitable choice is setting $\kappa = \bar{\alpha}+\bar{\beta}+1$ and $\lambda = 3$. This yielded the following counterplot in the β and α axis.



7.1 "Generating 5,000 random outcomes from the posterior distribution"

$$f_{\theta_i,\alpha_i,\beta_i|\boldsymbol{X}_i}\left(\theta_i, \alpha_i, \beta_i \mid \boldsymbol{x}_i\right)$$

Since our posterior distribution lacks an analytical form it will be difficult to generate samples from it. We can however use numerical methods like the metropolis algorithm to get good approximations.

Algorthmic overview of the metropolis algorithm:
1.
Initialization: Choose an initial state x_0 and a proposal distribution $q\left(x' \mid x\right)$ which suggests a new state x' given the current state x.
2.
For each iteration $t = 1, 2, \ldots, T$: a. Proposal: Sample a candidate x' from the proposal distribution $q\left(x' \mid x_t\right)$. b. Acceptance Probability: Compute the acceptance probability α as:

8 Sources

- Parameter Estimation for the Beta Distribution Claire Elayne Bangerter Owen Brigham Young University - Provo. 2008-11-10. https://scholarsarchive.byu.edu/cgi/viewcontent.cg
- https://www.youtube.com/watch?v=08TdhlGt170
- The Metropolis algorithm, https://www.algorithm-archive.org/contents/metropolis/metropolis.html